

Documentation & AI Usage Report

Project Execution Guide, Architectural Decisions, Limitations & AI Collaboration Disclosure

Author: Satvik Sharma | **Project:** LLD Practice Platform | **Repository:**
github.com/sharmasatvik159-pixel/LLD-Practice-Performance

1. How to Run the Project

Prerequisites: Node.js (v18+) and npm (v9+) installed. Git configured.

Step 1: Clone Repository:

git clone https://github.com/sharmasatvik159-pixel/LLD-Practice-Performance.git

Step 2: Backend Execution:

Open a terminal: cd server && npm install && npm run dev → Server starts on **http://localhost:5000**.

Step 3: Frontend Execution:

Open a second terminal: cd client && npm install && npm run dev → Client launches on **http://localhost:5173**.

Step 4: Automated Testing:

Run npm test inside the server directory to execute all 10 domain unit tests, 4 e2e problem scenarios, and 5 error injections (100% pass rate).

2. Key Architectural Decisions

- **Clean Architecture & Domain-Driven Design:** Business entities (Problem, Attempt, Submission, Evaluation) are written in pure JavaScript, completely decoupled from HTTP/Express concerns.
- **Guarded Finite State Machine:** Strict lifecycle transition rules (DRAFT → SUBMITTED → EVALUATING → COMPLETED) enforce domain integrity and reject out-of-order calls.
- **Strategy Pattern for Pluggable Evaluators:** Abstract Evaluator interface allows seamless switching between the *Rule-Based Evaluator* and *AI Evaluator* via the EvaluationService registry.
- **Structured 7-Field Form:** Guides candidate thought process systematically (Requirements → Assumptions → Classes → Responsibilities → Relationships → Patterns → Edge Cases).
- **Dependency Injection:** PracticeService receives ProblemService via constructor injection, enabling clean unit testing and mock isolation.

3. Known Limitations & Production Roadmap

Limitation in Current MVP	Architectural Rationale	Production Roadmap Solution
In-Memory Storage	Zero-friction local execution without database setup.	Implement Repository Pattern with PostgreSQL / Prisma ORM.
Heuristic AI Engine	Offline, 0ms latency, zero API key/cost dependency.	Integrate live LLM APIs (OpenAI / Claude) with structured JSON parsing.
Synchronous Pipeline	Sub-50ms execution gives immediate client response.	Migrate to async worker queue (BullMQ + Redis) with WebSockets.
Textual Representation	Simplifies candidate input and rubric parsing.	Add interactive Mermaid.js diagram editor and UML AST validator.

4. Comprehensive AI Usage Report

Tools & Models Utilized: Google DeepMind Antigravity / Gemini & Claude Coding Assistant.

Nature of Assistance: Agentic pair-programming spanning architecture design, domain model debugging, test automation scaffolding, and documentation generation.

Subsystem / Task	AI Contribution	Human Responsibility
System Architecture & DDD	40% (Pattern drafting, class outlines)	60% (Scoping, domain rules, final design sign-off)
Domain Models & State Machine	50% (Transition methods, error guards)	50% (Invariants review, lifecycle validation)
Evaluator Engines (Strategy)	60% (Scaffolding Rule & AI strategies)	40% (Rubric criteria definitions, scoring logic)
Frontend UI (React + Vite)	45% (Component layouts, CSS tokens)	55% (UX review, state wiring, user flow QA)
Automated Test Suite & E2E	70% (Scaffolding runner and HTTP tests)	30% (Defining test cases, edge case scenarios)
Documentation & PDF Reports	60% (ReportLab formatting scripts)	40% (Technical writing, trade-off justifications)

Human Oversight & Verification: All generated code was thoroughly reviewed, debugged, and verified through manual end-to-end browser walkthroughs and automated test execution. The human engineer retained complete intellectual ownership of architectural decisions, domain invariants, and product requirements.

Prepared by: Satvik Sharma | **Engineering Lead**